

Day 10 - Iterators, Generators, and Decorators

Day 10 — Iterators, Generators, and Decorators

Objectives

- Understand the actual mechanism a `for` loop uses internally — something you've been relying on since Day 2 without seeing how it works
- Write your own generator functions using `yield`, and understand exactly why they save memory
- Understand decorators as a direct, practical application of Day 4's closures
- Write your own decorators, including ones that accept their own arguments

Peeling back the curtain on `for` loops

Since Day 2, you've written many `for` loops without ever needing to know *how* Python actually pulls items out of a list, a string, or a dict one at a time. Today you'll learn the real mechanism — partly because it's genuinely useful to understand, and partly because it's the exact foundation that today's main topic, generators, is built directly on top of.

Two new vocabulary words: something is called **iterable** if it has a built-in method named `__iter__` that can produce an **iterator** — a separate, different kind of object whose entire job is to hand back exactly one value at a time, on demand, via its own `__next__` method, and to signal "there's nothing left" by raising a special exception called `StopIteration` once it's exhausted. A `for` loop is really just convenient shorthand for a pattern you could, in principle, write yourself:

```
my_list = [10, 20, 30]

iterator = iter(my_list)      # calls my_list.__iter__() behind the scenes, returning an iterator
while True:
    try:
        item = next(iterator)  # calls iterator.__next__(), asking for "the next value"
    except StopIteration:
        break                 # no more items -- exit the loop
    print(item)                # this is the loop "body" you'd normally write directly under `for`
```

This is exactly, precisely what `for item in my_list:` does for you automatically, hiding all of this machinery so you can just write the simple version. The reason this matters today: this exact same protocol — something with a `__next__` that can be asked, one at a time, "what's next?" — is precisely what a generator (below) provides, which is why generators plug directly into `for` loops with zero extra effort.

Generators — functions that hand back values one at a time, pausing in between

A **generator function** looks almost like an ordinary function, except somewhere in its body it uses the keyword `yield` instead of, or alongside, `return`. Calling a generator function does **not** immediately run its body top to bottom the way calling a normal function does — instead, it immediately hands you back a special

generator object, and the function's actual code only starts running, one small step at a time, as something asks it for its next value (which, again, a `for` loop does for you automatically).

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i      # hand back the CURRENT value of i, and PAUSE right here
        i += 1       # this line only runs the NEXT time something asks for another value

for number in count_up_to(5):
    print(number)    # prints 1, 2, 3, 4, 5 -- one at a time, as the loop asks for each one
```

Here's the part that takes a moment to fully click: each time execution reaches `yield i`, the function's entire state — every local variable, exactly where execution was — is frozen in place, and that current value of `i` is handed back to whoever asked. The **next** time something asks this generator for another value, execution resumes exactly where it left off (right after that same `yield` line), not from the very top of the function again. This is fundamentally different from a normal function, which always starts fresh from its very first line, every single time you call it.

Why bother? The concrete, practical reason: memory

Consider the difference between these two ways of getting the same ten million numbers:

```
numbers_list = list(range(10_000_000))    # builds and stores ALL 10 million numbers in memory RIGHT NOW

def numbers_generator(n):
    i = 0
    while i < n:
        yield i
        i += 1

numbers_gen = numbers_generator(10_000_000)  # this line runs INSTANTLY and uses almost no memory at all
```

`numbers_list` genuinely allocates space for ten million integers, all at once, whether or not you actually end up using all of them. `numbers_gen`, by contrast, holds essentially nothing in memory beyond "where I currently am" — it produces the next number only the instant something actually asks for it, and immediately forgets it again once it's been handed over. For ten million small integers this difference might not matter much on a modern computer, but the exact same idea applies just as well to something like reading through a multi-gigabyte log file line by line — a generator lets you process it a piece at a time without ever needing to fit the whole file in memory simultaneously.

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()    # hands back ONE line at a time, never holding the whole file in memory
```

An important limitation to know about: once a generator has been fully consumed (you've asked it for every value it has, or you stopped partway through and abandoned it), it's finished for good — you cannot rewind it or restart it. If you need to go through the same sequence of values more than once, you either call the generator *function* again to get a brand-new, fresh generator object, or, if the underlying data is genuinely small enough, just use an ordinary list instead.

Generator expressions — the generator version of a list comprehension

You met list comprehensions on Day 2. Swap the square brackets for parentheses, and you get the lazy, generator equivalent:

```
squares_list = [n * n for n in range(1_000_000)]    # computes and stores ALL one million squares
squares_gen = (n * n for n in range(1_000_000))    # computes NOTHING yet -- purely a plan, ready to go

total = sum(squares_gen)    # NOW, as sum() asks for each value in turn, the actual computation happens
```

Decorators — functions that wrap other functions with extra behavior

A **decorator** is a function that takes another function as input, and returns a brand-new function that adds some extra behavior *around* the original — all without needing to touch or rewrite the original function's own source code. This is built from ideas you already have every piece of: closures (Day 4) and **args/**kwargs* for forwarding arguments along unchanged (also Day 4).

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)    # actually calls the ORIGINAL function, with whatever is passed in
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f} seconds")
        return result
    return wrapper

@timer
def slow_add(a, b):
    time.sleep(0.1)
    return a + b

print(slow_add(2, 3))    # prints something like "slow_add took 0.10xx seconds", then 5
```

That *@timer* line, written directly above *def slow_add(...)*, is nothing more than convenient shorthand for writing *slow_add = timer(slow_add)* right after the function is defined — Python takes the freshly-defined *slow_add*, immediately hands it into *timer*, and replaces the name *slow_add* with whatever *timer* returns (here, *wrapper*). From that point on, whenever anyone calls *slow_add(...)*, they're actually calling *wrapper*, which does its own extra work (timing) and then, inside itself, calls the **original** *slow_add* function (kept alive as *func*, thanks to the closure).

You've very likely already **seen** decorators without knowing what they were — *@staticmethod*, *@classmethod*, and *@property* (built-in ones used with classes) are decorators, and if you go on to use popular tools like Flask (*@app.route(...)*) or pytest (*@pytest.fixture*, which you'll meet on Day 12), you'll see this exact *@something* pattern constantly. Understanding the mechanism today means none of those will ever feel like unexplainable magic.

``functools.wraps`` — a small but important detail

Without extra care, a decorated function quietly loses its original name and its docstring, because the name *slow_add* now actually points at *wrapper*, not at the original function:

```
print(slow_add.__name__)    # without care, this would print "wrapper" -- confusing in logs, debugger, etc.
```

The fix is to add one more decorator, from Python's built-in `functools` module, directly onto your own `wrapper` function — its whole job is to copy the original function's name, docstring, and other identifying details onto the wrapper:

```
from functools import wraps

def timer(func):
    @wraps(func)          # copies func's __name__, __doc__, etc onto wrapper
    def wrapper(*args, **kwargs):
        ...
    return wrapper
```

Make `@wraps(func)` a habit every time you write a decorator of your own, starting with today's exercises.

Decorators that take their own arguments

Sometimes you want to configure a decorator itself, like `@repeat(times=3)`. This requires one extra layer of nesting: a plain function that, when called, *returns* a decorator (rather than being a decorator directly):

```
def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def greet(name):
    print(f"Hello, {name}!")

greet("Ana")    # prints "Hello, Ana!" three separate times
```

Read this from the inside out to make sense of it: `repeat(times=3)` runs *first*, immediately, and its job is only to produce and hand back `decorator` (a plain function, now aware of `times=3` thanks to the same closure mechanism from Day 4). *Then*, that returned `decorator` gets applied to `greet`, exactly like any ordinary decorator from earlier in this lesson.

Exercises

Open `exercises.py`, fill in each `# TODO`, and run `python exercises.py`.